Examen Corregido 1

Pregunta 0.1
¿Que significa que un objeto sea una instancia de clase? Explica si dos instancias deben tener iguales valores en sus atributos
R= una instancia es un objeto que fue creado a partir de una clase. Todas las instancias de clase comparten la definición de atributos y metodos pero los valores guardados en sus atributos de instancia pueden ser distintos

Pregunta 0.2
¿Que diferencia hay entre un parametro y un argumento de un metodo ilustrala con una llamada sencilla
R= El parametro se declara directamente en la cabecera del metodo. El argumento es el valor especifico que se le da al metodo al realizar la llamada Ejemplo: Carga.(3)

Pregunta 0.3
¿Que es el ambito de una variable local? Da un ejemplo de donde se puede usar y donde deja de estar disponible
R= El ambito es donde el nombre de una variable es visible y se puede utilizar Por ejemplo una variable declarada dentro de un bloque if solo existe en ese mismo bloque y fuera de el no se puede usar

Pregunta 0.4
¿Que guarda una variable de tipo Arreglo? Explica que ocurre al asignarla a otra variable del mismo tipo
R= una variable de tipo arreglo guarda una referencia hacia el arreglo al asignarla a otra variable del mismo tipo se copia la referencia provocando que ambas apunten al mismo arreglo Cualquier cambio realizado en una se refleja en la otra tambien

Pregunta 0.5
¿Que hace un break dentro de un case de switch Clasico? Explica que puede ocurrir si falta
R= Un break es la salida del switch y si la instruccion falta ocurre un efecto de fall-through en donde no vuelve a comparar sus etiquetas

P1 d) Qué imprime?

Escribe la salida Exacta, una línea por renglon
Integer original = 7;
Integer copia = original;
int vuelta = 0;
do {
    Original = original + 2;
    System.out.println(original/2);
    vuelta++
} while (vuelta < 3);
String texto = "E" + copia + 1;
System.out.println(texto);
System.out.println((Double)(original/2));

Ejecucion	original	copia	vuelta	texto	Salida
Valores iniciales	7	7	0	—	—
Vuelta 1	9	7	1	—	4
Suma 2 e imprime division					
Vuelta 2	11	7	2	—	5
Suma 2 en imprime division					
Vuelta 3	13	7	3	—	6
Suma 2 en imprime division					
Fin v	13	7	3	"E71"	—
~~Finales~~					
Impresion final	13	7	3	"E71"	"E71" 6.0

* Al sumar lo 2 se hace la division
* No hay vuelta 3 ya que en la 3 Evalua < 3 vuelta < 3 (falso) por lo que concatena

P2 a) Compilar truena o funciona?
String codigo = null;
boolean igual = codigo.equals("133");
System.out.println(igual);

Compila pero truena, la responsable del error es la linea boolean igual = codigo.equals("133"); ya que se invoca un metodo ~~inexistente~~ utilizando una referencia con valor null y lanza null pointer exception

USUAL

R2 b)

```
double voltaje = 7.6;
int lectura = voltaje;
System.out.println(lectura);
```

R= No compila. Se da un error debido a int lectura= voltaje; ocurre ya que java no permite convertir automaticamente en doble a un int

R2 c)

```
Integer reserva = 4
int nueva = reserva + 3
System.out.Println(nueva);
```

R= Funciona e imprime 7. Java realiza automaticamente un Unboxing del Integer, extrayendo su valor primitivo sumando 4+3 = 7

R3 Modelo de: base, retos, justificación

Cada banda nace vacía con capacidad propia fija. El límite de seguridad común es 12 piezas. agregarPieza() suma una y consultarPiezas() responde cuantas hay. Sabemos que solo se llama agregarPieza() cuando ambos límites lo permiten

Base. Declara al menos dos atributos de instancia con tipos, uno estatic y una constante final (¿también static?) Completa el constructor y 2 metodos: cambio y consulta

Reto. ¿Cuál es final de instancia y cual static final: capacidad o límite de seguridad? Justifica una linea por decisión tipo, instancia/clase, final e mutable (constructor y metodos) Cuerpos de 1 a 3 líneas, sin estructuras de control ni main hay mas de un modelo correcto. Un elemento sin justificar vale la mitad.

Codigo —

```
Class banda {
Final int capacidad
static final int Limite_Seguridad = 12;
int piezas = 0;
Banda (int capacidad){ this.Capacidad = capacidad; }
void agregarPieza() { Piezas ++; }
int consultarPiezas(){ return piezas; }
}
```

USUAL

Justificacion B3

Atributo piezas: Instancia notable de tipo int iniciado en 0
Atributo capacidad: atributo final de instancia representa una capacidad propia y fija
Atributo Limite_Seguridad: atributo static final de tipo int debido a que es una constante de valor 12
Constructor: recibe el argumento de la capacidad y utilica el this para guardar ese valor
Metodo agregarPieza(): modifica el estado incrementando el contador (Piezas ++) y consultarPiezas() devuelve la cantidad actual del contador

B4 Codigo con Repeticion

Un equipo parte con carga unidades. En cada paso suma aporte
Devuelve cuantos pasos necesita - para alcanzar o superar neta usando while. Si ya llega, devuelve 0 carga y neta estan entre 0 y 10000; aporte entre 1 y 100. No usas una formula de division
static int pasosCarga(int carga, int aporte, int neta)
Ejemplo Caraguanta - retorna) No inventas ni lees del teclado
5, 4, 16 -> 3 · 16 ¿? 16 -> 0 · 20, 4, 16 -> 0 Escribe metodo completo
Codigo
static int pasosCarga(int carga, int aporte, int neta){
int pasos = 0;
while(carga < neta) { carga = carga + aporte;
pasos++;
} return pasos;
}
Explicacion
5, 4, 16 devuelve 3: inicia con carga 5 entra al ciclo porque 5 < 16 paso 1 carga sube a 9 paso 2 sube a 13 paso 3 sube a 17 como 17 supera 16 el ciclo termina devuelve 3

16, 4, 16 devuelve 0 inicia con carga 16 la condicion 16 < 16 es falsa de inmediato, el ciclo no se ejecuta

B5 Codigo con seleccion

Decide si en banco de prueba, puede arrancar. Recibe el porcentaje de bateria y si la tapa esta cerrada
Aplica la primera regla que corresponda, en el orden de la tabla. Usa if-else-if / else, con && y ||. No imprima ni lea del teclado

Static String permiso(int bateria, boolean tapaCerrada)
Regla o valor recibido
Bateria menor que 0 o mayor que 100 "invalida"
Tapa cerrada y bateria al menos 60 "normal"
Tapa cerrada y bateria al menos 20 "Ahorro"
Cualquier otro caso "Bloqueado"

Codigo:

```
Static String permiso(int bateria, boolean tapaCerrada) {
  if (bateria < 0 || bateria > 100) {
    return "Invalida";
  } else if (tapaCerrada && bateria >= 60) {
    return "normal";
  } else if (tapaCerrada && bateria >= 20) {
    return "Ahorro";
  } else {
    Return "Bloqueado"
  }
}
```

Explicacion

Devuelve "Invalida" si se recibe una bateria fuera de los limites logicos cubriendo el caso inicial con el operador logico

Devuelve "Normal" si pasa la validacion anterior y mediante un AND se cumple que la tapa esta cerrada y que la bateria es mayor igual a 60 (bateria >= 60)

Devuelve "Ahorro" a traves del else if si la tapa esta cerrada pero la bateria esta al 20% (bateria >= 20

Devuelve "Bloqueado" mediante un bloque else que abarca todos los casos restantes por ejemplo tapa abierta o menos del 20% de bateria